How the Recursive Global Solver Works

Chebyshev interpolation, Smolyak grids, and the two-branch expectation in `code/global/solver_recursive/`

Computational methods note

August 11, 2026

Abstract

This note explains, from the ground up and in plain terms, the three numerical ideas your recursive global solver in `code/global/solver_recursive/` is built from. **Chebyshev interpolation** lets us replace an unknown policy function by a short list of numbers. **Smolyak sparse grids** let us do that in six state dimensions without the point count exploding. And the **two-branch expectation** in `point_map.py` is how sovereign-default risk actually enters the banker's optimality conditions: the future is split into a no-default and a default continuation, weighted by a priced probability. I keep the maths self-contained and lead with intuition, with small worked examples you can check by hand, and a dedicated section shows exactly which functions get approximated and where the interpolation plugs into the solve loop. The last two sections are the ones you asked for specifically: how the "branch" is computed in your code (and why the older representative-branch version in `solver_pf/` got the sign wrong), and a careful analysis of *why the solve fails at the* μ=2 *grid while* μ=1 *works.*

Contents

1 Orientation: what runs where

Your `code/global/` tree contains two different ways of solving the same two-country model, and it is worth being clear which one this note is about.

- `solver_pf/` — the **perfect-foresight** solver. This is what `main.py` (the file you have open) runs: it stacks the whole T=200-period transition into one big system of equations and hits it with a damped Newton method (`transition.py`, `solvers.py`). Sovereign risk is handled by `risk_branch.py` using a single *representative* post-default economy. No Chebyshev, no Smolyak here.
- `solver_recursive/` — the **recursive global** solver. Instead of one long time path, it solves for *decision rules*: functions that map any state S to the equilibrium choices. Those functions are represented with Chebyshev polynomials on a Smolyak grid, and the expectations are done with a genuine two-branch quadrature. **This is the code this note explains.**

The recursive solver exists to fix a specific problem with the representative-branch approach: as `main.py` itself notes (`RUN_RISK_TWO_BRANCH = False`), that approximation produces an *expansionary* response to sovereign risk — the wrong sign. The recursive solver, using the real distribution over the default event, recovers the right sign (risk lowers output and consumption). We will see exactly why in §7.

2 What we are actually solving for

Start with the object we want. In a recursive equilibrium the "answer" is not a single number or a single time path — it is a set of *rules*, one for each equilibrium quantity, each a function of the current state. Your state is the six-vector defined in `state_grid.py` (`STATE_NAMES`):

$$S \;=\; (\,K_D,\; K_F,\; P_D,\; P_F,\; B_D,\; s\,),$$

the two capital stocks, the two banks' gross deposit obligations, the stock of risky D-government debt, and the risk factor s that sets the priced default probability. For each state the solver must return values for the seven "market-clearing" unknowns (`decision_rules.py`, `SOLVE7`),

$$(N_D,\, N_F,\, K'_D,\, K'_F,\, r_D^{\text{dep}},\, r_F^{\text{dep}},\, p),$$

plus a handful of read-off objects (banker valuations α, bond prices Q_b, consumption C). Each of these is a *function of the whole state* — a surface over a six-dimensional box.

Why can we not just linearize (Taylor-expand) around the steady state, as a perturbation method would? Two reasons. First, the banks face a leverage constraint that binds only *sometimes* — it is slack in normal times and clamps down in a crisis. That switch puts a *kink* in the true policy, and a Taylor polynomial around one point cannot see a kink somewhere else. Second, the entire point of the exercise is the behaviour *away* from the steady state, in the stressed region. So we need a *global* method: one that represents the whole surface, not just its slope at a point.

The recipe for that is the *projection method*: pick a flexible family of functions (here, Chebyshev polynomials), and choose the member of that family that makes the model's equations hold exactly at a finite set of sample states (the *collocation nodes*). Everything below is machinery for doing that well. Three questions drive it: *which functions and which sample points* (§3), *how to place sample points in six dimensions without needing millions of them* (§4), and *how to compute the expectations the equations contain* (§7).

3 Approximating a function of one variable

Forget economics for a moment. Suppose you have a smooth function f on the interval $[-1, 1]$ and you want to store it in a computer using only a few numbers, so that you can later evaluate it anywhere, cheaply and accurately. The plan: write f as a weighted sum of fixed "building-block" polynomials,

$$f(x) \;\approx\; p_n(x) \;=\; c_0\,T_0(x) + c_1\,T_1(x) + \cdots + c_n\,T_n(x),$$

and store just the weights $c_0, \ldots, c_n$. The building blocks T_k are the *Chebyshev polynomials*, and the reason they are the right choice is the subject of this section.

3.1 The Chebyshev polynomials

Here is the slickest definition. On $[-1, 1]$ every x can be written as $x = \cos\theta$ for some angle $\theta \in [0, \pi]$. Define

$$T_k(x) \;=\; \cos(k\theta) \qquad \text{where } x = \cos\theta.$$

That looks like trigonometry, not a polynomial — but it *is* a polynomial of degree k in x, because $\cos(k\theta)$ can always be rewritten using powers of $\cos\theta = x$. You do not have to take this on faith; the following recurrence builds them and is exactly the code in `state_grid.py`:

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_k(x) = 2x\,T_{k-1}(x) - T_{k-2}(x). \tag{1}$$

Worked example. Building the first few Chebyshev polynomials by hand.

Start from $T_0 = 1$ and $T_1 = x$ and turn the crank of (1):

$$\begin{aligned} T_2 &= 2x \cdot x - 1 = 2x^2 - 1, \\ T_3 &= 2x(2x^2 - 1) - x = 4x^3 - 3x, \\ T_4 &= 2x(4x^3 - 3x) - (2x^2 - 1) = 8x^4 - 8x^2 + 1. \end{aligned}$$

Sanity check against the trig definition: $\cos(2\theta) = 2\cos^2\theta - 1$, and substituting $x = \cos\theta$ gives $T_2 = 2x^2 - 1$. They agree. Notice each T_k stays between -1 and $+1$ on the interval (it is a cosine), and it wiggles more as k grows — T_k has exactly k roots in $[-1, 1]$. Those wiggles are what let a sum of them trace out any shape.

In code, (1) fills a table of $T_0, \ldots, T_n$ at a list of points:

```
def chebyshev_basis_1d(x, max_deg):
    T = np.empty((x.size, max_deg + 1))
    T[:, 0] = 1.0
    if max_deg >= 1:
        T[:, 1] = x
    for k in range(2, max_deg + 1):
        T[:, k] = 2.0 * x * T[:, k - 1] - T[:, k - 2]
    return T
```

Why use the recurrence instead of just storing f as $a_0 + a_1 x + a_2 x^2 + \ldots$ (ordinary powers)? Because the powers $1, x, x^2, x^3, \ldots$ start to look almost identical to each other on $[-1, 1]$ as the degree grows (they all flatten out), so solving for the coefficients a_k becomes numerically unstable — tiny rounding errors blow up. The Chebyshev polynomials, being disguised cosines, stay "spread out" and keep the computation stable.

3.2 Why the sample points matter: the Runge disaster

To pin down $n + 1$ coefficients we need the fit to match f at $n + 1$ sample points (nodes). The obvious choice — evenly spaced points — is surprisingly terrible. The textbook illustration uses $f(x) = 1/(1 + 25x^2)$, a perfectly smooth, innocent-looking bump. Fit a polynomial through evenly spaced points and, as you add more points, the fit does not get better — it develops wild oscillations near the ends of the interval that grow *without bound* (Figure 1, left). This is the *Runge phenomenon*.

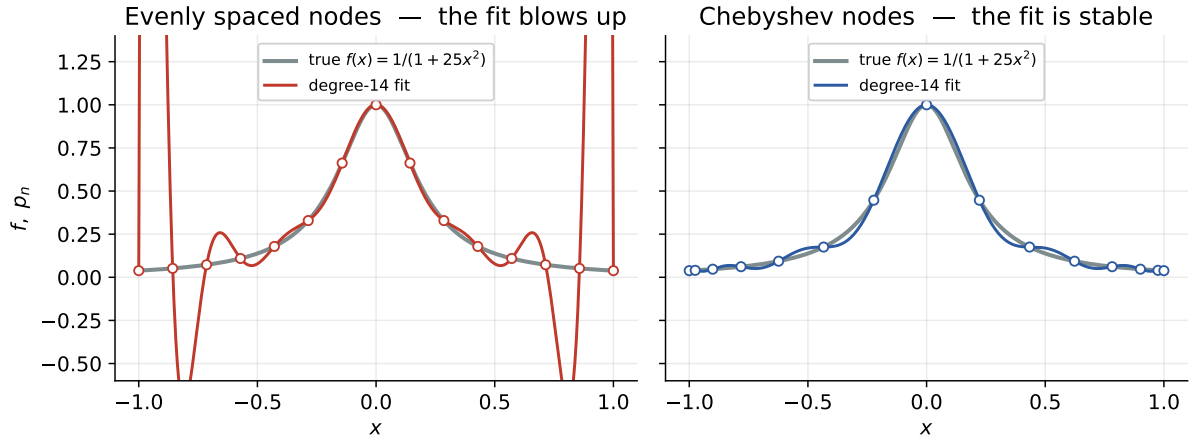

Figure 1: The same degree-14 fit of the same smooth bump. On evenly spaced nodes (left) the polynomial swings violently near ± 1; on Chebyshev nodes (right) it tracks the function. Only the placement of the dots differs.

The cure is to *cluster* the sample points toward the ends of the interval. The right clustering is given by the *Chebyshev nodes* (used throughout `state_grid.py`):

$$x_k \;=\; -\cos\!\Big(\frac{\pi k}{m-1}\Big), \qquad k = 0, \ldots, m-1. \tag{2}$$

There is a lovely picture behind this formula (Figure 2): space m points evenly *around* a semicircle, then drop each straight down onto the horizontal axis. The shadows they cast are the Chebyshev nodes, and because a semicircle is steep near its ends, the shadows bunch up near ± 1 — exactly the clustering that defeats Runge.

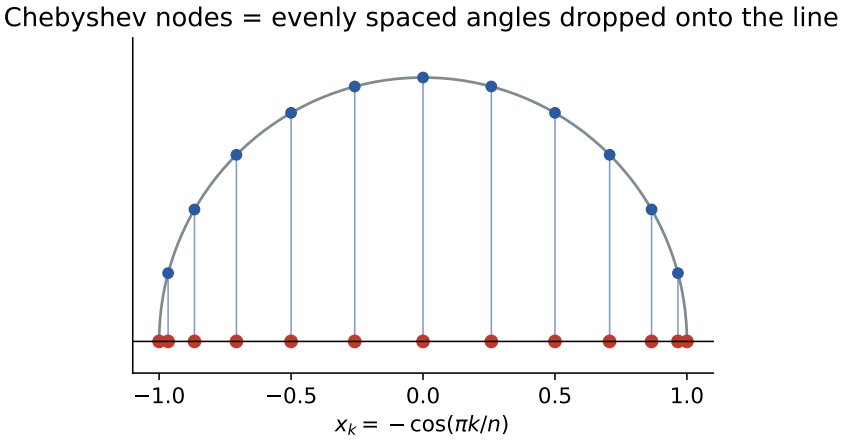

Figure 2: The Chebyshev nodes are evenly spaced angles on a semicircle, projected onto the line. The projection crowds them toward the endpoints.

There is a precise sense in which this choice is nearly the best possible. A number called the *Lebesgue constant* measures how much worse interpolation can be than the theoretically ideal fit. For evenly spaced nodes this number grows *exponentially* with the degree (catastrophe); for Chebyshev nodes it grows only like $\log n$ — so slowly that in practice interpolating on Chebyshev nodes is about as good as it is possible to do. You do not need the formula; the takeaway is: *Chebyshev nodes turn a potentially explosive problem into a tame one.*

3.3 Fitting is just solving a small linear system

Say we have decided on the degrees $0, \dots, n$ and the nodes $x_0, \dots, x_n$. "Fit f" means: find the coefficients c so that $p_n(x_k) = f(x_k)$ at every node. Writing that out for each node gives $n+1$ linear equations in the $n+1$ unknowns c,

$$\underbrace{\begin{pmatrix} T_0(x_0) & \cdots & T_n(x_0) \\ \vdots & & \vdots \\ T_0(x_n) & \cdots & T_n(x_n) \end{pmatrix}}_{\text{basis matrix } \Phi} \begin{pmatrix} c_0 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ \vdots \\ f(x_n) \end{pmatrix}, \qquad \text{i.e. } \Phi\, c = f. \tag{3}$$

The matrix Φ depends only on the grid, never on f. So the code computes it once, factorizes it once (an "LU factorization," just bookkeeping that makes repeated solves fast), and reuses it forever after. In `state_grid.py` that factor is `self._lu`; the two workhorse methods are `fit` (given values f, solve for c) and `eval` (given c, evaluate p_n anywhere). Every rule in `decision_rules.py` is stored this way, as a vector of point values plus its fitted coefficients.

Worked example. A three-point fit you can do on paper.

Take three Chebyshev nodes on $[-1, 1]$: from (2) with $m = 3$ they are $x = -1,\ 0,\ +1$. The building blocks are $T_0 = 1$, $T_1 = x$, $T_2 = 2x^2 - 1$. Evaluate them at the three nodes to get the basis matrix, and match a function with values a, b, c at $x = -1, 0, 1$:

$$\Phi = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 0 & -1 \\ 1 & 1 & 1 \end{pmatrix}, \qquad \Phi \begin{pmatrix} c_0 \\ c_1 \\ c_2 \end{pmatrix} = \begin{pmatrix} a \\ b \\ c \end{pmatrix}.$$

Solving the 3×3 system by hand gives the tidy answer $c_0 = \frac{b}{2} + \frac{a+c}{4}$, $c_1 = \frac{c-a}{2}$, $c_2 = \frac{a+c}{4} - \frac{b}{2}$. Try it on $f(x) = e^x$, whose values are $a = e^{-1} = 0.368$, $b = 1$, $c = e = 2.718$. You get $p_2(x) = 1.000 + 1.175\,x + 0.543\,x^2$. This reproduces e^x exactly at the three nodes and to within a few percent in between (worst error about 0.07 near $x = \frac{1}{2}$). That is the whole idea in miniature: *three numbers stand in for a function.* Add a couple more Chebyshev degrees and the error drops to machine precision — which is the next point.

3.4 How accurate is it? Smoothness decides

The accuracy of a Chebyshev fit depends almost entirely on how *smooth* the target function is. Two regimes matter (Figure 3):

- If f is **very smooth** (infinitely differentiable, no kinks), the error falls *geometrically* — each extra degree multiplies it by a constant factor well below one. This is called *spectral accuracy*: you reach machine precision with a couple of dozen degrees. The smooth curve in Figure 3 is a straight line on a log scale, the signature of geometric decay.
- If f has a **kink** (a corner, where the slope jumps), the error falls only *algebraically* — painfully slowly, like $1/n$ or $1/n^2$ — and the fit develops small ripples near the corner (the same family of wiggles as the Runge oscillations, here called Gibbs oscillations).

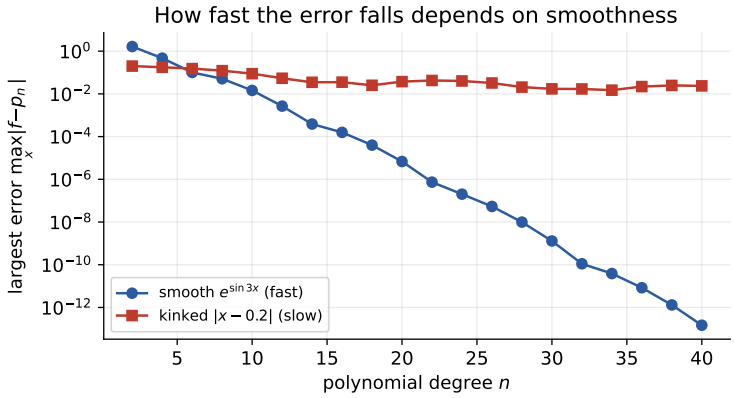

Figure 3: Largest fit error versus polynomial degree, on Chebyshev nodes. A smooth target (blue) races down to machine precision; a kinked target (red) crawls. The occasionally-binding leverage constraint puts a kink in the model's true policy, so the red curve is the one to keep in mind.

This is not a minor footnote for us: the bank leverage constraint is occasionally binding, which puts a genuine kink into the true policy functions. So in the region where the constraint switches on, a polynomial basis is fighting its worst case. That single fact is behind much of §6 and all of §8.

3.5 The one thing you must never do: extrapolate

A Chebyshev fit is trustworthy *inside* $[-1, 1]$. Outside it, the polynomials do not just get a bit worse — they explode. A degree-n Chebyshev polynomial grows roughly like $\left(|x| + \sqrt{x^2 - 1}\right)^n$ once $|x| > 1$, which is enormous.

Worked example. How fast extrapolation blows up.

The degree-10 block T_{10} satisfies $T_{10}(1) = 1$ at the edge of the interval. Step just outside:

$$T_{10}(1.05) \approx 12, \qquad T_{10}(1.2) \approx 252, \qquad T_{10}(1.5) \approx 7{,}560.$$

A 20% overshoot of the box edge magnifies that one building block by a factor of 250; if its coefficient is not tiny, the "fit" returns nonsense — a negative consumption, say, which then feeds into an expectation and produces `NaN`. (These numbers are computed directly from the recurrence (1).)

This is why the recursive solver *clips* every evaluation point back into the box before interpolating. You see it in `point_map.py` wherever the continuation is evaluated, e.g. `cont.eval_all(d_next, cont.grid.clip(Sn))` — `clip` projects any stray next-period state back onto the box boundary, so the rule is never asked to extrapolate. It is a deliberate, visible guard, not a silent patch.

4 Six dimensions without the explosion: Smolyak grids

Section 3 handled one variable. Our state has six. The naive extension is a *tensor grid*: put m Chebyshev nodes on each axis and take every combination. That is m^6 points. Even a coarse $m = 5$ gives $5^6 = 15{,}625$ collocation points — and at every one of them the solver must solve a nonlinear system and compute an expectation, on every iteration. Push to $m = 9$ and it is over half a million. This geometric blow-up with the number of dimensions is the famous *curse of dimensionality* (Figure 4, grey line). Tensor grids are hopeless past three or four states.

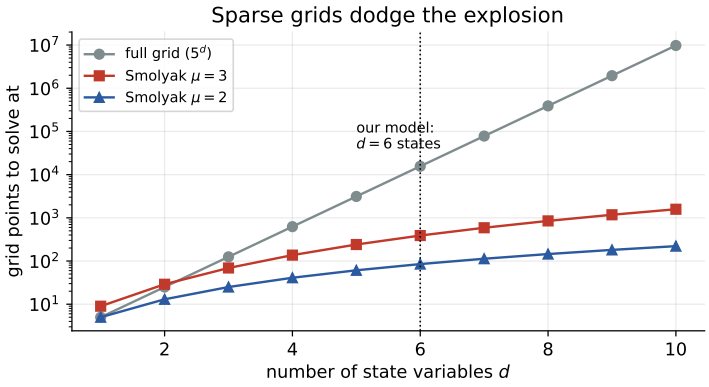

Figure 4: Points needed versus number of states. The full tensor grid explodes geometrically; the Smolyak grids grow gently. At our six states the Smolyak grid needs 13 points (μ=1), 85 (μ=2) or 389 (μ=3) — against tens of thousands for a comparable tensor grid.

4.1 The idea: keep the cheap combinations, drop the expensive ones

Smolyak's insight is that most of a tensor grid is wasted. The points that resolve fine detail in *many* dimensions *at once* (the deep interior of the grid) are hugely numerous but contribute little; the useful points resolve detail in one or two dimensions at a time. Smolyak keeps only the latter. Concretely it works with *levels*: level-i resolution in one dimension uses m_i nested Chebyshev nodes,

$$m_1 = 1, \quad m_i = 2^{\,i-1} + 1 \;\; (i \geq 2) \implies m = 1, 3, 5, 9, \ldots$$

"Nested" means every node of one level is reused at the next — level 2's three points contain level 1's single point, and so on (`_level_points` in `state_grid.py`). A combination of per-dimension levels is written as a multi-index $\mathbf{i} = (i_1, \ldots, i_d)$. A full tensor grid would allow every combination; Smolyak keeps only the *cheap* ones, those whose total level is small:

$$\text{keep } \mathbf{i} \quad \Longleftrightarrow \quad \sum_{j=1}^{d} (i_j - 1) \;\leq\; \mu. \tag{4}$$

The single number μ ("mu" in the code) is the *approximation level*: $\mu = 0$ is one point, $\mu = 1$ adds one level of detail along each axis, $\mu = 2$ adds pairwise interactions, and so on. This is exactly the `_multi_indices` routine, which enumerates the kept combinations.

Worked example. Building the $\mu = 2$ grid in two dimensions — all 13 points.

Take $d = 2$, $\mu = 2$. Rule (4) keeps every $\mathbf{i} = (i_1, i_2)$ with $(i_1 - 1) + (i_2 - 1) \leq 2$. There are six such combinations. For each, we take the tensor product of the *new* points that level introduces (level 1 adds $\{0\}$; level 2 adds the endpoints $\{-1, 1\}$; level 3 adds $\{\pm 0.707\}$):

$(1,1) \to \{0\} \times \{0\}$	1 point (the centre)
$(1,2), (2,1) \to \{0\} \times \{-1,1\}$ & $\{-1,1\} \times \{0\}$	4 points (axis ends)
$(1,3), (3,1) \to \{0\} \times \{\pm.707\}$ & $\{\pm.707\} \times \{0\}$	4 points (axis interior)
$(2,2) \to \{-1,1\} \times \{-1,1\}$	4 points (the corners)

Total: $1+4+4+4 = 13$ points — which is exactly what `SmolyakGrid([-1,-1],[1,1],mu=2).n` returns. Figure 5 plots them, coloured by which combination they came from. Notice the shape: a dense cross along the axes plus the four corners, with the expensive dense *interior* of a full grid simply absent. That gap is the saving.

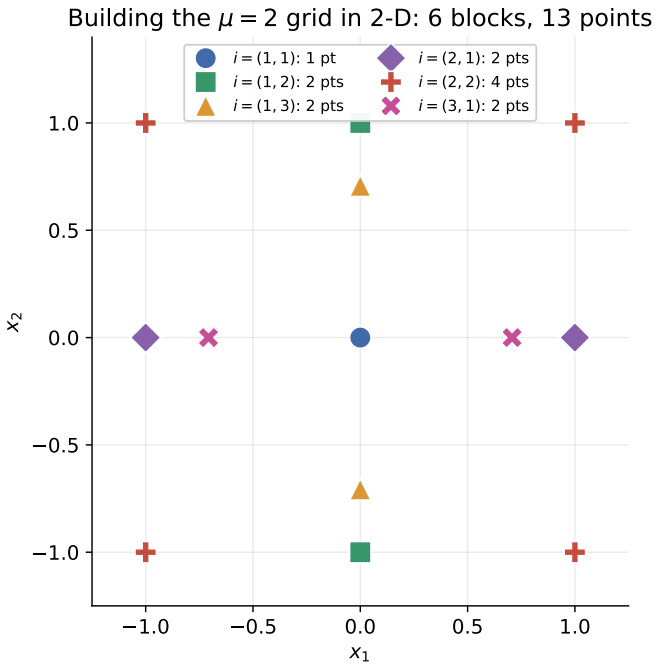

Figure 5: The 13 points of the two-dimensional $\mu = 2$ Smolyak grid, coloured by the multi-index block that contributes them (the worked example above). This is the actual `state_grid.SmolyakGrid` construction.

The same rule in higher dimensions and higher levels gives the sparse crosses of Figure 6. The bookkeeping detail that makes it all work: points and polynomial degrees are added in lock-step (`_new_points` alongside `_new_degrees`), so the basis matrix Φ is square and invertible, and "fit" is again a single solve of $\Phi c = f$ — just with 13, 85, or 389 rows instead of 3.

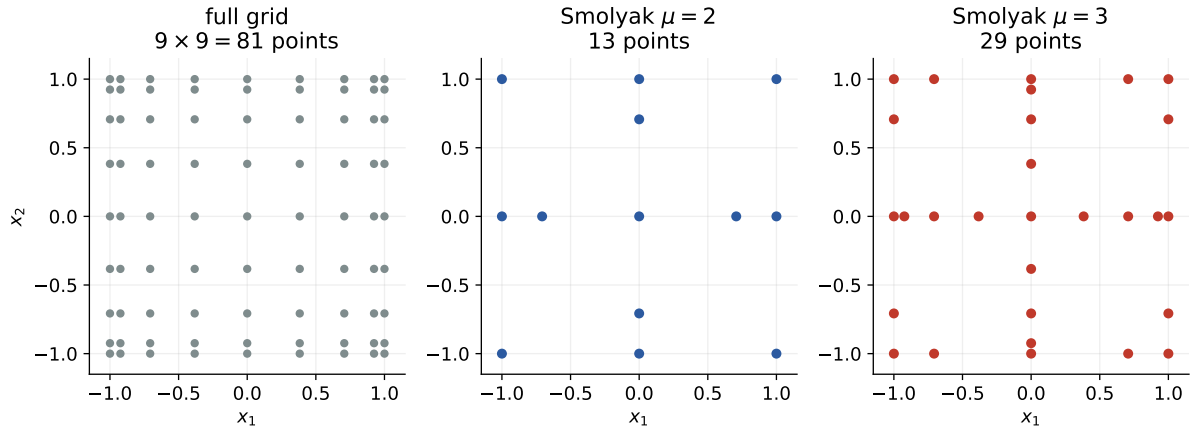

Figure 6: Full tensor grid (left) versus Smolyak at $\mu = 2$ and $\mu = 3$. The sparse grids keep the axes and a few cross terms and discard the dense middle.

4.2 What μ buys, and what it costs

A higher μ represents more interactions between states: $\mu = 1$ is essentially *separable* (it can bend along each axis but captures almost no cross-effects — how the response along one state depends on another); $\mu = 2$ adds all pairwise cross-terms; $\mu = 3$ adds triples. More faithful, but more points (13, 85, 389 at $d = 6$), and — as §8 is entirely about — more of those points sit in awkward corners of the state space. Two more features of `state_grid.py` matter later:

- `build_state_box` places the box around the steady state, with per-state half-widths (`k_band`, `p_band`, `b_band`) and the s range set so the priced default probability spans about 0.01% to 3%. The bands are deliberately narrow for K (capital barely moves) and wider for P, B (the movers).
- `mu_vec` allows an *anisotropic* grid — a different level per state. You can ask for cross-terms only in the dimensions where the constraint boundary actually moves (s, P, B) and keep the nearly fixed capital dimensions at level 1. This is the natural lever to try against the $\mu = 2$ failure, and §8 comes back to it.

5 Connecting the tools to the model: what we fit, and how well

Sections 3–4 were deliberately abstract: how to approximate "a function f," and how to place sample points in many dimensions. This section ties that back to the model — *which* functions we actually fit, *why* the fit is needed at all, and *how well* it converges for the particular functions at hand. It is the bridge between the machinery and the economics, and it sets up the $\mu = 2$ analysis in §8.

5.1 The functions we fit are the decision rules

The things we run Chebyshev over are the model's *decision rules* — the equilibrium quantities written as functions of the state. The code (`decision_rules.py`) lists them in two groups:

- `SOLVE7` — the seven market-clearing unknowns $N_D, N_F, K'_D, K'_F, r_D^{\text{dep}}, r_F^{\text{dep}}, p$;
- `DERIVED` — eight quantities read off the optimality conditions: the banker valuations α_D, α_F, the bond prices Q_{bD}, Q_{bF}, consumption C_D, C_F, and deposit holdings A_D, A_F.

That is 15 quantities. Each is stored *per default regime* $d \in \{0, 1\}$ (`RuleSet`), so we are really approximating $15 \times 2 = 30$ separate scalar functions, each one a surface over the six-dimensional state box. Every one is represented the same way: its values at the Smolyak points, plus the Chebyshev coefficients fitted to them.

Here is a concrete way to feel the compression. At $\mu = 2$ each rule is 85 numbers, so the **entire** global equilibrium of the model is just $30 \times 85 = 2{,}550$ numbers (at $\mu = 1$ it is $30 \times 13 = 390$). Everything downstream — every impulse response, every stressed-state allocation — is reconstructed from those numbers by evaluating the interpolant. The solver's whole job is to find the right 2,550 numbers.

5.2 The problem we solve is a fixed point in those rules

What makes finding them non-trivial is that the rules are self-referential. The equilibrium conditions at a state S involve *next period's* rules, because a bank choosing today must forecast what it will do tomorrow. So the rules we are solving for appear on both sides: the rules agents use to look ahead must equal the rules that describe their behaviour. That is a *fixed-point* problem, and `recursive_main.time_iteration` solves it by repeated substitution (Figure 7): guess the rules, treat them as tomorrow's behaviour (the "continuation"), re-solve every grid point today, refit, and repeat until nothing moves.

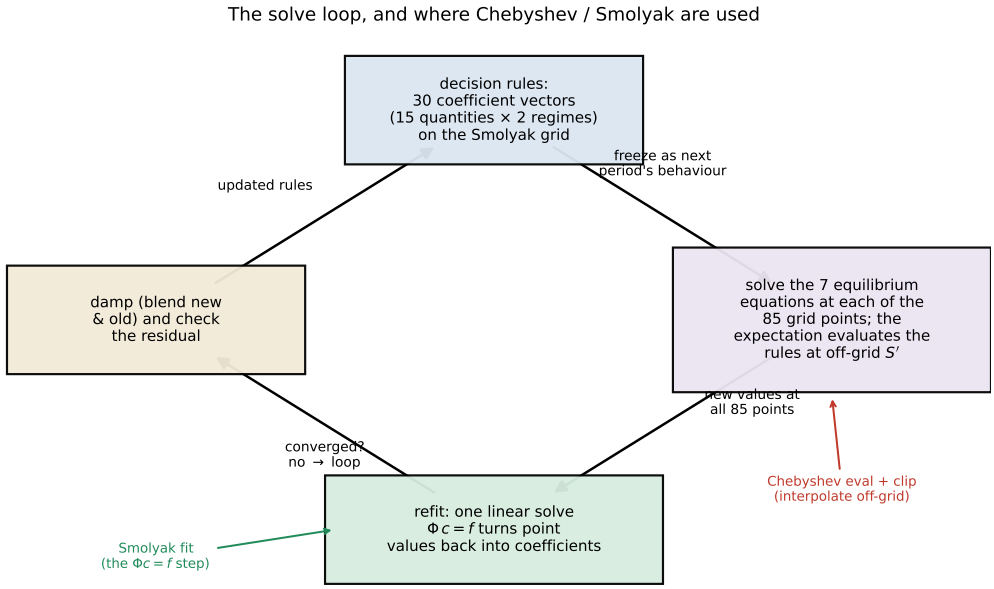

Figure 7: The time-iteration loop, with the two places the Chebyshev/Smolyak machinery is load-bearing marked in colour: evaluating the frozen rules at off-grid next-states S' inside the expectation (*red*), and refitting solved point values into coefficients (*green*).

5.3 Why the interpolant is needed at all

This is the heart of the connection, and the single best answer to "how do Chebyshev and Smolyak plug into the model." When we solve the equilibrium at one grid-point state S (Figure 7, right box), the banker's expectation looks one step ahead to the states S' that today's

choices lead to — one S' for each Gauss–Hermite shock draw, in each default branch (§7). Those next-states are **almost never grid points**; they are arbitrary interior states. To ask "what will banks do at S'?" we must read the continuation rules *off the grid* — and that is exactly, and only, what a global interpolant provides. In the code it is the one line `cont.eval_all(d', cont.grid.clip(Sn))`: a matrix product of the Chebyshev basis at S' against the stored coefficients, with `clip` guarding the extrapolation cliff of §3.5.

So the three ingredients earn their places precisely here. Without a *functional* representation you would know the rules only at the 85 grid points and could not form the expectation at the reachable S' at all. *Chebyshev* makes reading between the grid points accurate and cheap. *Smolyak* keeps the number of grid points affordable in six dimensions, so that "solve at every point, every iteration" is feasible. The two coloured callouts in Figure 7 are the two spots this matters: interpolating the continuation off-grid, and the refit.

5.4 One fit serves all thirty rules

A practical efficiency worth noting: because the basis matrix Φ depends only on the grid, the *same* factorized Φ (the LU factor of §3.3) fits every rule. `grid.fit` takes a whole array of point values and returns coefficients in a single solve, and `eval_all` evaluates all rules at a point with one basis matrix. The per-iteration cost is therefore dominated by the 85×2 pointwise nonlinear solves, not by the fitting — the interpolation layer is nearly free once Φ is factorized.

5.5 How well it converges — for these particular functions

Finally, "convergence" means two different things here, and keeping them apart is essential for §8.

1. **Approximation convergence** — how close the *best* Chebyshev fit of a given rule gets to the true rule, as we raise the level μ. This is the smoothness story of §3.4 applied to the decision rules. Over most of the state box — where the leverage constraint is comfortably slack or comfortably binding — the rules are smooth, and the fit is near-spectral: a modest μ already captures them and raising it drives the error down fast. But every one of these rules has a *kink* where the constraint switches on (the $\max\{\cdot, 0\}$ in the closed-form multiplier). Right at that crease the rule is only continuous, not smooth, so — exactly like the red curve in Figure 3 — the approximation converges slowly there and a higher-degree fit rings. In one line: *the functions we fit are mostly smooth with one crease, and the crease is where all the trouble concentrates.*

2. **Solver convergence** — whether the time-iteration loop actually *reaches* its fixed point. This is a different question. Even if each rule *could* be represented well, the loop might not get there: it might oscillate, or settle on the wrong equilibrium. The $\mu = 1$ grid converges in this second sense; the $\mu = 2$ grid does not.

The honest consequence, flagged earlier: at $\mu = 1$ the magnitudes are only *indicative*, because a near-separable fit cannot capture how the responses interact across states — but the *sign and mechanism* are trustworthy, because every rule it uses is read at well-behaved, converged points. Sharpening the magnitudes needs a fit that resolves both the crease and the cross-state curvature. That is exactly what $\mu = 2$ was supposed to buy — and §8 is the story of why, here, it cannot.

6 The numerical risks, in plain terms

Before the branch and the $\mu = 2$ analysis, here is the short list of ways this kind of solver goes wrong, and the guard for each. They all reappear in §8.

1. **Extrapolation (§3.5).** Evaluating a rule outside its box returns garbage. Guard: `grid.clip` before every interpolation in `point_map.py`.

2. **Kinks and Gibbs wiggles (§3.4).** The occasionally-binding constraint is a kink; a global polynomial fits it only slowly and rings near it. In the recursive solver the multiplier is handled by Bocola's *closed form*, $\mu = \max\{1 - \mathbb{E}[\Omega] R\, n/(\lambda \cdot \text{assets}),\, 0\}$ (`point_map.py`), capped just below 1 so it stays finite when a deep-crisis net worth goes small. (The perfect-foresight solver in `solver_pf/` instead uses Fischer–Burmeister smoothing; same economics, different numerical dressing.)

3. **Bad corners poisoning a global fit.** Because the Chebyshev basis is *global*, every coefficient depends on *all* node values (that single $\Phi c = f$ solve). So if the solve fails at a few awkward corner states, freezing them at stale values still corrupts the fitted surface *everywhere*. This is the central villain of §8. Guard (partial): the sweep *retains* the previous iterate at any point that does not clear (`recursive_main._sweep`, `keep_tol`) — which stops a single blow-up but cannot stop the contamination.

4. **Reaching a deep crisis from a calm start.** The post-default regime is far from the steady-state cold start, so a solver launched from the steady state may not find it. Guard: a *homotopy* — lower the recovery rate in steps $0.85 \to 0.70 \to 0.55 \to$ target, re-solving each (`recursive_experiment.solve_recursive`).

5. **A slow outer loop.** The rules are found by *time iteration* — guess the rules, use them as next period's behaviour, re-solve, repeat. Its convergence speed is tied to the model's persistence (≈ 0.98 for capital), so it crawls and can wobble. Guards: damping (blend new and old), and measuring convergence on rule *values*.

7 The branch computation

Now the piece you asked about: how "the branch" is computed. The word refers to how the model handles the fork in the road each period — next period the sovereign either defaults ($d' = 1$) or does not ($d' = 0$) — inside the expectations that appear in the banks' optimality conditions. Your repo contains *two* ways of doing this, and the difference between them is the whole reason the recursive solver exists.

7.1 The economic object: an expectation over a fork

A banker's optimality condition (an Euler equation) says, roughly, "the price of an asset today equals the expected discounted value of its payoff tomorrow." The expectation runs over next period's shocks *and* over the default fork. For any payoff X',

$$\mathbb{E}\big[X'\big] = \underbrace{\sum_{\text{shock draws } j} w_j}_{\text{average over shocks}} \Big[\underbrace{(1 - p^d)\, X'^{(d'=0)}}_{\text{no-default branch}} + \underbrace{p^d\, X'^{(d'=1)}}_{\text{default branch}} \Big], \tag{5}$$

where the probability of default is a logistic function of the risk factor (`state_grid.default_prob`),

$$p^d(s) = \frac{1}{1 + e^{-s}}. \tag{6}$$

At your calibration (`s_process_params`) the resting value is $s^\star = -6.91$, giving $p^d \approx 0.1\%$, and a two-standard-deviation risk shock lifts s to about -3.9, i.e. $p^d \approx 2\%$ (Figure 8, right). The crucial modelling point is that (5) is a genuine *probability-weighted average over two possible futures* — a distribution, not a single representative number. Raising s shifts weight onto the low-payoff default branch.

7.2 The wrong way (representative branch, `solver_pf/`)

The perfect-foresight solver that `main.py` runs handles this with a *representative* branch (`solver_pf/risk_bran` it computes one stand-in "feared default" economy once and reuses it at every date, pairing its low payoffs with a high marginal valuation to price a risk premium. It is a reasonable-sounding shortcut, but it collapses the distribution to a single point, and in this model that gets the sign *wrong*. With only one frozen default economy, capital ends up looking like the safe place to hide, so banks lever *into* capital when risk rises and output *expands* — the opposite of the data and of Bocola's result. That is precisely why `main.py` keeps it switched off (`RUN_RISK_TWO_BRANCH = False`) and why the recursive solver was written.

7.3 The right way (two-branch quadrature, `point_map.py`)

The recursive solver evaluates (5) honestly. The "average over shocks" is done by *Gauss–Hermite quadrature* — the standard, exact tool for averaging against a bell curve. With the technology and spending shocks held fixed for this experiment, the only continuous shock is the risk innovation ϵ_s, and 7 Gauss–Hermite nodes suffice (`expectations.gh_nodes`). Each node is pushed one step forward through the AR(1) law for s to a next-period state; then the default fork doubles it into a no-default and a default copy. The whole thing lives in `point_map.point_residuals`: the helper `_cont(d_next)` evaluates next period's rules in regime d', and `_E(v0,v1)` takes the probability-weighted average (Figure 8, left):

```
w_nd, w_def = wq * (1.0 - pd), wq * pd     # shock weights x branch probs
def _E(v0, v1):                            # two-branch expectation
    return float(np.dot(w_nd, v0) + np.dot(w_def, v1))
```

The single most important idea is what `_cont` does: the "default economy" is *not* a separate object bolted on — it is **the same decision rules, evaluated at a reachable next-period state**, in the $d' = 1$ coefficient set. The bad state is somewhere the rules already describe; you just arrive there with probability p^d. Three things differ between the two branches, all visible in `point_map.py`:

- **Which rules.** Each rule has a separate coefficient set per regime, `coef[name][d]`; the default branch reads the $d' = 1$ set, the no-default branch the $d' = 0$ set.
- **The bond payoff.** In the default branch the D-bond is written down to its recovery value (`recovery_rate_D`); this haircut, weighted by p^d, is what pulls the bond price down when risk rises.
- **The discount factor.** The no-default branch discounts with the bank's usual kernel $\Omega^0 = \beta_{\text{inter}}(f + (1-f)\alpha')$; the default branch uses an income-based factor $\Omega^1 = \Lambda^1(f + (1-f)\alpha')$ with $\Lambda^1 = \beta_{\text{inter}}(Y^{d'=1}/Y^{d'=0})^{-\sigma}$ — higher marginal value when default-state income is low. This is what makes the low default payoff *extra* costly.

Worked example. The two-branch expectation with numbers.

Take a bond that pays 1 next period if there is no default and, after a 55% haircut, 0.45 if there is (recovery 0.45). At the resting risk level $p^d = 0.001$ the expected payoff is $(1 - 0.001) \cdot 1 + 0.001 \cdot 0.45 = 0.99945$ — essentially 1. Now hit the economy with the $+2\sigma$ shock, $p^d = 0.02$:

$$\mathbb{E}[\text{payoff}] = (1 - 0.02) \cdot 1 + 0.02 \cdot 0.45 = 0.989.$$

The *average* payoff falls about 1.1%. But the price falls by *more*, because the default branch also carries the higher discount factor $\Omega^1 > \Omega^0$. Suppose $\Omega^0 = 1$ and, with income down in

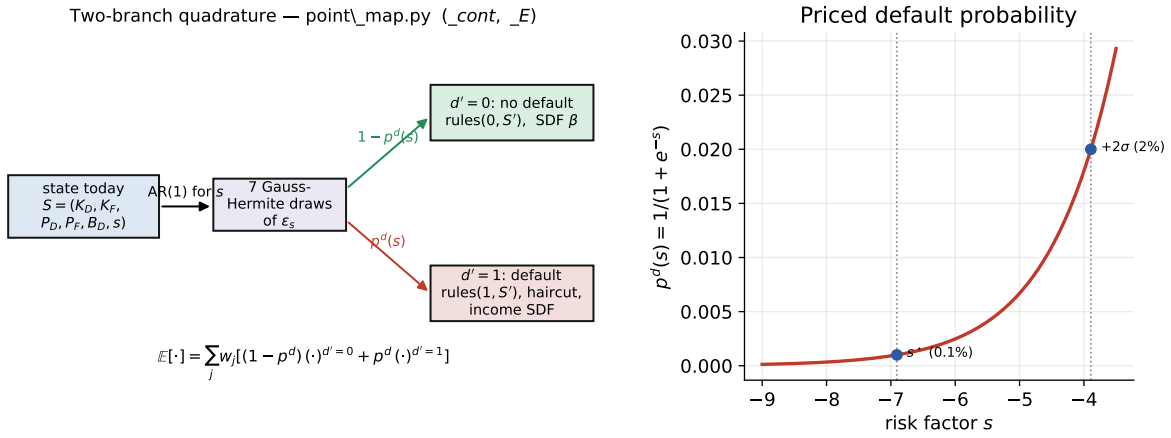

Figure 8: *Left:* the two-branch quadrature in `point_map.py`. The current state goes forward through the AR(1) for s to 7 Gauss–Hermite draws; each splits into a no-default continuation (weight $1 - p^d$, ordinary discounting) and a default continuation (weight p^d, bond haircut and income discounting); the expectation is their weighted sum. *Right:* the logistic default probability, with the resting point $s^\star$ (0.1%) and a $+2\sigma$ risk shock (2%) marked.

the default state, $\Omega^1 = 1.3$. The bond-Euler numerator the code forms, $\mathbb{E}[\Omega \cdot \text{payoff}]$, goes from $0.98 \cdot 1 \cdot 1 + 0.02 \cdot 1.3 \cdot 0.45 = 0.9917$ at $p^d = 0.02$ against a no-risk value of 1 — and the same rise in p^d lowers bank net worth, which raises the multiplier μ in the denominator of the bond-price formula `Q_bD_new = E_Om_payD / (E_Om_D*(1+rdep) + lambda*mu)`. Both moves push Q_{bD} down. *That* is the pass-through: higher $s \Rightarrow$ more weight on the haircut branch $\Rightarrow$ lower bond price $\Rightarrow$ mark-to-market loss on bank balance sheets $\Rightarrow$ tighter constraint and wider lending spreads $\Rightarrow$ lower investment and output. With the distribution represented honestly, the sign comes out right: elevated default risk *lowers* Y_D and C_D, persistently (the result read off by `recursive_experiment.impact_table` and `persistence_irf`).

---

8 Why the solve fails at $\mu = 2$ but works at $\mu = 1$

This is the question you flagged. The recursive experiment currently runs on the $\mu = 1$ grid by default (`solve_recursive(..., mu=1)`), and the code comment records the reason bluntly: at $\mu = 2$ "the $d = 1$ regime does not converge on the wide box (joint residual ~ 0.7, sign flips expansionary)." Here is what is going on and why it is not a bug you can patch away — it is the same global-basis obstacle documented for the standalone Bocola solve in `docs/bocola2016_replication.md`.

8.1 The symptom

Recall the two grids at six states: $\mu = 1$ has 13 points, $\mu = 2$ has 85. When you solve both default regimes:

- $\mu = 1$ **converges** to a small residual and gives the economically correct, contractionary sign. Its weakness is only that, being nearly separable (§4), it cannot represent how the responses interact across states — so its magnitudes are "indicative," but its *sign and mechanism are right.*
- $\mu = 2$ **does not converge.** The no-default ($d = 0$) block is fine, but the joint two-regime solve leaves a worst residual around 0.7 — six orders of magnitude above the 10^{-6}

target — and, worse, the implied risk response flips to *expansionary*, the very artifact the recursive solver was built to eliminate.

8.2 Why $\mu = 1$ gets away with it

Look again at which points each grid uses (§4). The $\mu = 1$ grid is just the *axes*: the centre, plus a step along each state one at a time. Every one of those 13 points sits close to the steady state (the box bands are tight, 2%–8%), so every point is economically well-behaved — positive net worth, a constraint that is at most gently binding, a nonlinear solve that clears cleanly. There are no "two things extreme at once" points. The homotopy walks the $d = 1$ regime down to the full haircut without ever meeting a state it cannot solve. Clean inputs everywhere give a clean fit and the loop converges. The price, as §5.5 said, is that a near-separable grid cannot represent how the responses interact across states — so the magnitudes are indicative, though the sign and mechanism are right.

8.3 The potential reasons $\mu = 2$ will not converge

Going from $\mu = 1$ to $\mu = 2$ adds exactly the points $\mu = 1$ omits: the *cross-corner* nodes, where two states are extreme at the same time (the $(2, 2)$ block of the worked example, the four corners of Figure 5). In six dimensions these are combinations like *low capital together with high risk*, or *high deposit obligation together with low surviving debt*. Adding them sets off not one problem but a stack of reinforcing ones. Here they are in plain terms, ordered roughly by how much the code's own diagnostics blame them.

1. **The constraint barely binds, so there are two nearby answers and the fine grid grabs the wrong one.** *(The deepest reason; the diagnostics confirm it.)* At the steady state the leverage multiplier is tiny, $\mu_{ss} \approx 0.02$ — the constraint is only just switched on, sitting right next to the $\max\{\cdot, 0\}$ kink. That close to the switch, the model has *two* valid equilibria almost on top of each other: a *binding* one, where a bank that cannot lever up responds to risk by de-risking (output falls, the right sign), and a *slack* one, where the bank substitutes into capital (output rises, the wrong sign). They are separated by a sliver. The coarse $\mu = 1$ grid is too blunt to "see" the slack answer, so the solver stays on the binding one. The finer $\mu = 2$ grid — more points, packed closer, with a root-finder launched from more places — keeps slipping onto the slack branch. That slip *is* the expansionary sign it reports. *Plainly:* two correct-looking answers sit almost on top of each other, and the fine grid keeps grabbing the wrong one.

2. **A few corners cannot be solved at all, and a global fit spreads their error everywhere.** *(Confirmed: the code retains failed points, and the fit is global.)* In the already-defaulted regime the 55% haircut has crushed net worth on arrival. At a cross-corner that *also* has, say, high deposit obligations, the bank is near insolvency — μ is pinned at its cap, net worth is tiny or negative — and the pointwise solver finds no sensible equilibrium. The code sensibly *retains* the previous value there rather than crash (`_sweep`, `keep_tol`). But the Chebyshev basis is *global*: every coefficient is a blend of **all** 85 point values (the single $\Phi c = f$ solve). So a handful of frozen, wrong corner values leak into every coefficient, and therefore into the fitted surface *everywhere* — including the calm interior where we read the answer. The interior then "solves" against a poisoned forecast, and the joint residual can never fall below ~ 0.7. *Plainly:* one wrong entry in a shared spreadsheet formula throws off every cell.

3. **The point solver is local and starts from one guess.** *(Structural.)* Each grid-point solve is a local root-finder (`scipy.root`) launched from the steady-state / warm-start guess. The deep-recession $d = 1$ solution can have a tiny "catchment" — you only land

on it if you start nearby. $\mu = 2$ adds many corner states far from any good guess, so more of them find no root, or the wrong one. *Plainly:* you are feeling for a light-switch in the dark from where you happen to stand, and the fine grid adds rooms you cannot reach from there.

4. **A polynomial cannot fold sharply, so it ripples at the crease.** *(Structural; §3.4.)* The true rules have a kink where the constraint turns on. A smooth polynomial forced to bend sharply overshoots and wiggles nearby (the Gibbs ripples). Those wiggles can nudge a neighbouring point to the wrong side of the $\max\{\cdot, 0\}$ switch, flipping it between the binding and slack regimes from one iteration to the next, so it never settles. $\mu = 2$ has more points near the crease, so more of them wobble. *Plainly:* drawing a sharp corner with a springy ruler — press harder and it bounces.

5. **The two regimes are solved against each other, so errors echo.** *(Structural.)* The $d = 0$ and $d = 1$ rules share the continuation: tomorrow's default-state rules feed today's no-default expectation and the other way round. A single bad $d = 1$ corner therefore contaminates $d = 0$ too, and back again. The joint solve is stiffer than either regime alone — and it is precisely the joint step where the residual sticks. *Plainly:* two people copying each other's homework; one mistake spreads both ways.

6. **The box cannot be both wide enough and clean.** *(Structural.)* Capital is almost a random walk (persistence ≈ 0.98), so the states the model truly visits form a wide, tilted cloud. A box wide enough to hold it has economically impossible corners; a box narrow enough to avoid them clips the continuation. Whatever box you choose, $\mu = 2$ populates its corners — and in $d = 1$ those corners are the near-insolvent states of reason 2. *Plainly:* the net must be big to catch the fish, but every extra metre of net drags in junk.

7. **Retaining failed points is a bandage, not a cure.** *(Structural; follows from 2.)* Freezing a point that will not solve keeps the run alive, but a frozen value is still a value that the global fit — and so the interior's forecast — treats as truth. It stops a crash; it does not stop the contamination.

8. **The outer loop is slow and can limit-cycle at the crease.** *(Structural.)* Time iteration's speed is tied to the model's persistence (≈ 0.98), so it crawls; and at the kink the affected points can oscillate between binding and slack across iterations without ever settling. Damping softens this but does not remove it. *Plainly:* a sluggish thermostat that keeps overshooting.

Reasons 1 and 2 are the load-bearing ones — the knife-edge multiplicity and the global-fit corner poisoning — with 3–8 as reinforcing structural difficulties. And they are not independent: the poisoned corners (2) push interior points toward the slack branch (1), and the crease wiggles (4) do the same, so the failure feeds on itself.

8.4 Why the sign flips, specifically

The flip to an *expansionary* response is not a separate bug — it is the fingerprint of reasons 1 and 2. Recall from §7 that the two-branch mechanism gives the contractionary sign *only when the default-branch continuation is a genuine recession*: the default state must have low income and a low capital return, so that the high discount factor Ω^1 multiplies a low payoff and makes risk costly. Both a slip onto the slack branch (reason 1) and a poisoned $d = 1$ continuation (reason 2) destroy that: capital reappears as a safe haven, banks lever *into* it when risk rises, and output expands — exactly the representative-branch artifact the recursive solver was built to kill. So seeing the expansionary sign is how you *know* the solve has drifted onto the wrong branch or poisoned its corners.

8.5 What would actually fix it

The levers line up with the reasons, and several are one-line changes the code already supports.

- **Bind a little harder in the calibration — attacks reason 1, the root fix.** *(A direction the diagnostics point to.)* The whole knife-edge exists because $\mu_{\text{ss}} \approx 0.02$ is barely positive. Lowering λ_K (or tightening the leverage target) so that μ_{ss} sits comfortably around 0.05–0.10 pushes the binding equilibrium away from the slack one; the binding branch becomes the unique, robust answer and the fine grid can no longer slip onto the wrong one. The cost is a small step away from Bocola's very small estimated friction — a modelling choice, not a bug.
- **Read the rules instead of re-solving — attacks reason 1 at read time.** The impact and IRF routines already evaluate the converged rules at their own policy values (`read_at`) rather than launching a fresh root-finder, precisely so the read stays on the binding branch instead of slipping to the slack one. Keeping every downstream read on the rules (never re-solving) is the cheap safeguard.
- **Refine only the states that move — attacks reasons 2 and 6, cheaply.** `build_state_box` accepts `mu_vec`: keep the near-fixed capital dimensions at level 1 and give level 2 only to (s, P, B), the states where the constraint boundary actually moves. This buys the cross-terms you want *without* creating the low-capital corners that blow up in $d = 1$. It is the cheapest first experiment, and the hook is already there.
- **Use a local basis — attacks reason 2 at its root.** Replace the global Chebyshev basis with a local one (splines / finite elements), so a hard corner's error stays local instead of leaking through the fit. This is the route the Bocola replication notes single out, having hit the identical wall (there the residual floored around 5×10^{-3} on a rotated grid; here, on the wider two-country $d = 1$ box, it is far worse at ~ 0.7).
- **Shape the box to the cloud — attacks reason 6.** A PCA-rotated box (used in the standalone Bocola solve) cuts the infeasible-corner fraction sharply, so fewer nodes sit on impossible states in the first place.
- **Extend the homotopy — attacks reason 3.** The recovery-rate ladder that makes the near-steady-state $d = 1$ solve work could be paired with a *box-widening* homotopy, growing $\mu = 2$'s reach only as the solution stabilizes.

Honest summary: $\mu = 1$ gives a converged, correctly-signed answer whose magnitudes are indicative; a converged $\mu = 2$ answer is not a matter of more iterations but of *moving off the knife-edge* (calibration) and/or *decoupling the hard corners from the interior* (a different representation). The two first things to try are the anisotropic `mu_vec` grid and the bind-harder calibration, because both are one-line changes the code already supports.

9 Putting it together

The full solve (`recursive_experiment.solve_recursive`) chains the pieces:

1. build the Smolyak box around the steady state and cold-start every rule at its steady-state value (`RuleSet.from_ss`);
2. calibrate the household budget anchors so the steady state is an exact rest point (`calibrate_household_ar`
3. time-iterate the no-default ($d = 0$) rules to convergence;

4. warm-start the default ($d = 1$) rules from $d = 0$ and walk the recovery rate down by homotopy;

5. refine both regimes jointly;

6. read the pass-through: the impact response as p^d rises (`impact_table`) and the decay of a one-off risk shock (`persistence_irf`).

Each sweep of the time iteration (`recursive_main.time_iteration`) freezes the current rules as next period's behaviour, solves the seven market-clearing unknowns at every grid point and regime, reads off the banker valuations and household aggregates, then damps and refits. It is the per-point image of the same equations the perfect-foresight solver stacks — the difference is that here the answer is a reusable *policy*, and the expectation over the default fork is genuine.

Summary

Three tools, one caveat. *Chebyshev interpolation* turns each policy function into a short, stable list of coefficients, near-optimal for smooth targets but explosive if you extrapolate and slow at kinks. *Smolyak grids* carry that into six dimensions for a handful of points instead of thousands, by keeping only the low-total-level combinations of nested Chebyshev levels. The *two-branch expectation* in `point_map.py` prices sovereign risk honestly, as a probability weight on a default continuation that is the same rules read at a reachable state — which is what gets the sign right where the representative branch got it wrong. The caveat runs through all of it: a global polynomial basis couples every collocation point to every other, so the method is only as good as the worst state you ask it to represent. That is why the box is kept narrow, why every evaluation is clipped, why failed points are retained rather than trusted — and, ultimately, why $\mu = 2$ stalls while $\mu = 1$ converges: the extra cross-corner points $\mu = 2$ introduces are states the $d = 1$ regime cannot solve, and a global basis cannot stop their contamination from spreading.

Where to read the code

Concept	File / function (`code/global/`)
Chebyshev recurrence, nodes	`solver_recursive/state_grid.py`: `chebyshev_basis_1d`, `_level_points`
Smolyak combination rule	`solver_recursive/state_grid.py`: `_multi_indices`, `SmolyakGrid`
Fit / eval (cached LU solve)	`solver_recursive/state_grid.py`: `fit`, `eval`, `clip`
State box, anisotropy	`solver_recursive/state_grid.py`: `build_state_box`, `mu_vec`
Per-regime rule storage	`solver_recursive/decision_rules.py`: `RuleSet`, `SOLVE7`
Two-branch expectation	`solver_recursive/point_map.py`: `_cont`, `_E`
Closed-form multiplier μ	`solver_recursive/point_map.py` (Bocola Prop. 1)
Default probability	`solver_recursive/state_grid.py`: `default_prob`, `s_process_params`
Time iteration, retain guard	`solver_recursive/recursive_main.py`: `time_iteration`, `_sweep`
Homotopy, $\mu = 1$ default, IRFs	`solver_recursive/recursive_experiment.py`
Representative branch (the "wrong" way)	`solver_pf/risk_branch.py`; toggled off in `main.py`

References and further reading

- L. Bocola (2016), "The Pass-Through of Sovereign Risk," *Journal of Political Economy* 124(4) — the economics the two-branch expectation implements.
- D. Krueger and F. Kubler (2004), *J. Economic Dynamics & Control* 28 — the nested Chebyshev–Smolyak construction the code follows.

- K. Judd, L. Maliar, S. Maliar and R. Valero (2014), *J. Economic Dynamics & Control* 44 — anisotropic Smolyak levels (the `mu_vec` idea). https://bfi.uchicago.edu/wp-content/uploads/Judd-Maliar-Valero-1.pdf
- L. N. Trefethen (2013), *Approximation Theory and Approximation Practice*, SIAM — Chebyshev interpolation, the Runge phenomenon, and spectral convergence, at a readable level.
- J. Brumm and S. Scheidegger (2017), *Econometrica* 85 — adaptive/local sparse grids, the corner-decoupling route out of the $\mu = 2$ obstacle.
- Internal: `docs/bocola2016_replication.md` — the identical global-basis corner obstacle, analyzed for the standalone solve.